

Scalable Claims Processing Automation and Intelligent Fraud Detection Using Big Data and Machine Learning

A Capstone Project Report submitted by

G. Nageswar Rao (Reg. No. 221FA23002)

V. Amarnath (Reg. No. 221FA23021)

Ch. Amrutha (Reg. No. 221FA23044)

P. Rohith (Reg. No. 221FA23045)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Data Science

Under the Supervision of

Dr. Sk Fathimabi

Professor, Department of Advanced Computer Science & Engineering



VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Department of Advanced Computer Science and Engineering

School of Computing and Informatics

Vignan's Foundation for Science, Technology & Research
(Deemed to be University), Andhra Pradesh – 522213, India

March 2026

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	The Big Data Challenge	5
1.3	Objectives	5
1.4	Organisation of the Paper	6
2	Literature Review	6
2.1	Statistical and Rule-Based Fraud Detection	6
2.2	Machine Learning Approaches	6
2.3	Big Data Frameworks for Insurance Analytics	7
2.4	Claim Automation and Intelligent Routing	7
2.5	Identified Research Gaps	7
3	Dataset Description and Exploratory Analysis	8
3.1	Dataset Overview	8
3.2	Feature Categories	9
3.3	Class Distribution and Imbalance	9
3.4	Missing Values	9
3.5	Descriptive Statistics of Key Features	9
3.6	Feature Correlations Relevant to Fraud	10
4	System Architecture and Methodology	10
4.1	High-Level Architecture	10
4.2	Data Preprocessing Pipeline	11
4.2.1	Column Removal	11
4.2.2	Date Feature Engineering	11
4.2.3	Missing Value Imputation	12
4.2.4	Label Encoding	12
4.2.5	One-Hot Encoding	12
4.2.6	Feature Scaling	12
4.2.7	Class Imbalance Correction (SMOTE)	12
4.3	Machine Learning Models	12
4.3.1	Logistic Regression (Baseline)	12
4.3.2	Decision Tree	13
4.3.3	Random Forest	13
4.3.4	Gradient Boosting	13
4.3.5	Model Selection Criterion	13

4.4	Complexity Scoring and Claim Routing	13
4.5	Algorithm	14
5	Big Data Integration	15
5.1	Motivation for a Distributed Architecture	15
5.2	Apache Spark Distributed Processing	15
5.2.1	Architecture	15
5.2.2	Distributed Preprocessing Pipeline	15
5.2.3	Distributed Training Throughput	17
5.3	Apache Kafka Real-Time Streaming Integration	17
5.4	HDFS Storage and Data Partitioning	18
5.5	Big Data Infrastructure Summary	19
6	Results and Analysis	19
6.1	Experimental Setup	19
6.2	Model Performance Comparison	19
6.3	Confusion Matrix Analysis	20
6.4	ROC Curve Analysis	20
6.5	Complexity Score Distribution	21
6.6	Routing Decision Breakdown	21
6.7	Fraud Probability Distribution	22
6.8	Feature Importance Analysis	22
6.9	API Response Time	23
7	Discussion	23
7.1	Interpretation of Results	23
7.2	Comparison with Existing Work	24
7.3	Limitations	24
7.4	Ethical and Regulatory Considerations	24
8	Conclusion and Future Work	25
8.1	Conclusions	25
8.2	Future Directions	25

List of Figures

1	High-level system architecture for claims processing automation.	11
2	ROC curves for all classifiers on the held-out test set.	21
3	Distribution of claim complexity scores on the test set. Claims with score > 40 are routed for human review.	21
4	Predicted fraud probability distribution across the test set. Bimodal shape con- firms good model calibration.	22

List of Tables

1	Summary of Key Related Works	8
2	Class Distribution in the Dataset	9
3	Descriptive Statistics of Selected Numerical Features	10
4	Training Throughput: Single-Node vs. Spark Cluster	17
5	Big Data Infrastructure Stack	19
6	Model Performance on the Test Set (Fraud Class)	20
7	Confusion Matrix — Gradient Boosting (Test Set)	20
8	Claim Routing Decision Summary (Test Set)	22
9	Top 10 Features by Gradient Boosting Importance (Gini)	23
10	FastAPI Inference Endpoint Latency (100 Concurrent Users)	23

Abstract

The auto-insurance industry processes tens of millions of claims annually, and the ever-growing volume, velocity, and variety of claims data strain conventional single-node processing pipelines. This paper presents a scalable, end-to-end framework for *Claims Processing Automation and Fraud Detection* that marries classic machine learning with a Big Data architecture centred on Apache Spark. Four supervised classifiers—Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting—are trained and evaluated on a real-world insurance claims dataset comprising 15,420 records and 39 features. Class imbalance is addressed with the Synthetic Minority Over-sampling Technique (SMOTE), and the best model is selected on the basis of ROC-AUC score. A rule-based complexity scorer assigns each incoming claim a numerical priority score; claims above the threshold are routed for human review, while the remainder follow an automated straight-through processing pathway. The Spark-powered distributed pipeline reduces model training wall-clock time by approximately $5.2\times$ on a four-worker cluster compared with the single-node scikit-learn baseline. The Gradient Boosting classifier achieves the highest AUC of 0.92, an F1-score of 0.87 for the fraud class, and a precision of 0.89, outperforming all baselines. A FastAPI micro-service exposes the inference endpoint, and a Streamlit dashboard provides real-time routing analytics. The contributions of this work include (i) an end-to-end reproducible pipeline, (ii) a complexity-based intelligent routing module, (iii) integration of Spark MLlib for distributed training, and (iv) a full-stack deployment architecture suitable for enterprise adoption.

Keywords: auto-insurance fraud detection, claims processing automation, Apache Spark, Spark MLlib, Gradient Boosting, SMOTE, ROC-AUC, Big Data, claim routing, intelligent triage.

1. INTRODUCTION

1.1. Background and Context

The global insurance sector is one of the largest and most data-intensive industries in the world. According to Swiss Re Sigma reports, global insurance premiums exceeded USD 6.3 trillion in 2022, and the associated claims volume runs into hundreds of millions of cases per year. In the auto-insurance segment alone, a single mid-sized insurer may adjudicate upwards of 500,000 claims annually. Each claim demands intake processing, document verification, fraud screening, damage assessment, settlement computation, and customer communication—tasks that have historically been performed manually by human adjusters.

Manual adjudication is inherently slow, error-prone, and inconsistent. Industry surveys indicate that the average claim takes 7–14 working days to resolve, with complex cases frequently exceeding 30 days. Human evaluators are susceptible to fatigue-induced errors, subjective biases, and knowledge asymmetries that manifest as inter-adjuster inconsistency. Moreover, the

cost of operating large adjuster work-forces is substantial: administrative expenses typically account for 20–30% of an insurer's combined ratio.

Compounding these operational challenges is the growing sophistication of insurance fraud. The Coalition Against Insurance Fraud estimates that fraud costs the U.S. property/casualty sector approximately USD 45 billion annually, with auto insurance constituting the largest segment. Fraudulent schemes range from simple exaggeration of genuine damage ("soft fraud") to fully staged accidents and phantom-vehicle collisions ("hard fraud"). Traditional rule-based fraud screens, which operate on manually curated indicator lists, are easily circumvented by adaptive fraudsters and generate unacceptably high false-positive rates.

1.2. The Big Data Challenge

Contemporary insurance operations generate data at a pace and in a variety that challenge conventional single-server analytics. The problem exhibits the canonical "three Vs" of Big Data:

- **Volume:** An insurer may accumulate terabytes of historical claims, policy records, telematics logs, and third-party data over a decade of operations. Training a robust fraud detector on this corpus is infeasible on a single machine.
- **Velocity:** New claims arrive continuously. A streaming inference pipeline must score incoming claims in near-real time (sub-second latency) to meet service-level agreements (SLAs).
- **Variety:** Claims data are heterogeneous, spanning structured relational tables, free-text accident narratives, scanned PDFs, digital photos of vehicle damage, telematics time-series, and third-party API feeds.

Apache Spark [7] has emerged as the de-facto standard for Big Data machine learning, providing fault-tolerant, in-memory distributed computation over Hadoop-compatible storage. Spark MLlib extends the core framework with a rich library of scalable ML algorithms, including Logistic Regression, Random Forest, and Gradient Boosted Trees, all of which are natively parallelised across a cluster.

1.3. Objectives

This work pursues the following specific objectives:

1. Design and implement a preprocessing pipeline that handles missing values, categorical encoding, feature scaling, and class-imbalance correction (SMOTE).
2. Train and compare four ML classifiers—Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting—under both single-node scikit-learn and distributed Spark MLlib settings.

3. Develop a rule-based complexity scorer that converts model outputs and domain heuristics into a numerical priority score to drive intelligent claim routing.
4. Integrate the end-to-end pipeline with Apache Spark for distributed batch training and Apache Kafka for streaming inference.
5. Expose the inference logic through a FastAPI REST endpoint and visualise results on an interactive Streamlit dashboard.
6. Benchmark the Spark pipeline against a single-node baseline in terms of training throughput and latency.

1.4. Organisation of the Paper

Section 2 reviews related work across fraud detection, Big Data frameworks, and claims automation. Section 3 describes the dataset and exploratory analysis. Section 4 presents the system architecture and methodology in detail. Section 5 elaborates the Big Data integration layer. Section 6 reports experimental results and analysis. Section 7 discusses findings, limitations, and implications. Section 8 concludes the paper and outlines future directions.

2. LITERATURE REVIEW

2.1. Statistical and Rule-Based Fraud Detection

The seminal survey by Bolton and Hand [1] established the foundational taxonomy of insurance fraud detection techniques. They distinguish between *supervised* methods—which require labelled historical fraud examples—and *unsupervised* anomaly detection, which identifies statistical outliers without prior labels. Their work highlighted that, even in the early 2000s, the class-imbalance problem (fraud rates typically 1–5%) severely degraded classifier performance when accuracy was used as the primary metric.

Viaene et al. [2] applied Bayesian Networks and Decision Trees to automobile insurance claims, demonstrating that probabilistic graphical models capture feature dependencies (e.g., the correlation between high repair costs and short policy tenure) that flat feature vectors miss. Their best Bayesian model achieved an accuracy of 72% on a holdout set. However, the model was computationally expensive for inference on large claims populations and was not designed for real-time deployment.

2.2. Machine Learning Approaches

Phua et al. [3] published a comprehensive survey of data mining methods for fraud detection, emphasising the need for cost-sensitive learning in insurance settings where the cost of a missed fraud (false negative) greatly exceeds the cost of a false alarm. They benchmarked

Naïve Bayes, Decision Trees, and Neural Networks, finding that ensemble methods consistently outperformed single learners on imbalanced datasets.

The introduction of SMOTE by Chawla et al. [4] addressed the imbalance problem by generating synthetic minority-class samples in feature space rather than simple over-sampling. SMOTE has since become the standard pre-processing step for fraud detection benchmarks.

Gradient Boosting Machines (GBM), popularised by Friedman [5], have repeatedly demonstrated state-of-the-art performance on tabular fraud detection tasks. Their sequential, additive nature allows the ensemble to progressively reduce residual errors, excelling on the non-linear decision boundaries that characterise fraud patterns.

2.3. Big Data Frameworks for Insurance Analytics

Dean and Ghemawat [6] introduced MapReduce as the foundational distributed computation paradigm, enabling batch processing of petabyte-scale datasets across commodity hardware clusters. While MapReduce is robust for ETL workloads, its disk-bound execution model incurs prohibitive latency for iterative ML training.

Zaharia et al. [7] addressed these limitations with Apache Spark, whose Resilient Distributed Dataset (RDD) abstraction keeps intermediate computation results in memory across iterations. Spark MLlib [8] extends this to a high-level ML pipeline API (analogous to scikit-learn's Pipeline class) that operates over distributed DataFrames. In practice, Spark achieves 10–100× speed-ups over MapReduce-based ML on standard benchmarks.

For streaming use cases, Apache Kafka [9] provides a distributed, fault-tolerant publish-subscribe messaging system capable of handling millions of events per second with end-to-end latencies in the low-millisecond range. Spark Structured Streaming can consume Kafka topics directly, providing a unified batch-and-streaming API.

2.4. Claim Automation and Intelligent Routing

Carcillo et al. [10] extended prior fraud detection work to a streaming, concept-drift-aware setting, demonstrating that models trained on historical data degrade over time as fraudsters adapt. Their SCARFF framework periodically retrains on recent windows of labelled data. However, the work did not address operational claim routing beyond the binary fraud/legitimate decision.

Fawcett [11] formalised ROC curve analysis and established the Area Under the Curve (AUC) as a threshold-independent performance metric that is robust to class imbalance—a property that makes it the preferred evaluation criterion for fraud classifiers.

2.5. Identified Research Gaps

Based on the review above, the following gaps motivate the present work:

1. Most existing studies focus exclusively on improving *model accuracy* rather than operationalising the model within an end-to-end claims workflow.
2. There is limited work on *hybrid* approaches that combine ML predictions with rule-based heuristics for complexity scoring and intelligent routing.
3. Few papers demonstrate a *scalable deployment architecture* integrating Spark, Kafka, and a REST API suitable for enterprise adoption.
4. Streaming inference under concept drift remains a partially open problem for auto-insurance fraud detection.

Table 1 summarises the key works and their limitations.

Table 1: Summary of Key Related Works

Reference	Technique	Metric	Limitation
Bolton & Hand (2002) [1]	Statistical, Anomaly Detection	Accuracy	Low scalability
Viaene et al. (2002) [2]	Bayesian Networks, DT	Accuracy 72%	No real-time routing
Phua et al. (2010) [3]	Ensembles, Cost-sensitive	F1	Survey only
Chawla et al. (2002) [4]	SMOTE over-sampling	F1, AUC	Pre-processing only
Carcillo et al. (2018) [10]	Streaming ML	AUC	No routing logic
Zaharia et al. (2016) [7]	Apache Spark	Throughput	Not domain-specific
Fawcett (2006) [11]	ROC Analysis	AUC	Evaluation only

3. DATASET DESCRIPTION AND EXPLORATORY ANALYSIS

3.1. Dataset Overview

The dataset used in this study is a publicly available auto-insurance claims dataset comprising **15,420 records** and **39 features** before preprocessing. Each record represents a single insurance claim. The binary target variable `fraud_reported` takes the value Y (fraudulent) or N (legitimate), mapped to 1 and 0 respectively.

3.2. Feature Categories

The features span four broad categories:

1. **Policyholder demographics:** Age, education level, occupation, annual income, insured relationship.
2. **Policy attributes:** Policy number, bind date, annual premium, policy deductible, coverage type, umbrella limit.
3. **Incident details:** Incident date, type (single-vehicle, multi-vehicle, parked), severity (trivial damage, minor damage, major damage, total loss), number of vehicles involved, number of witnesses, bodily injuries.
4. **Claims attributes:** Total claim amount, injury claim, property claim, vehicle claim, police report availability.

3.3. Class Distribution and Imbalance

The target class is highly imbalanced: **24.7%** of claims are fraudulent and **75.3%** are legitimate. This mirrors realistic industry fraud rates. The imbalance necessitates special handling during training to prevent the classifier from trivially predicting the majority class. Table 2 summarises the distribution.

Table 2: Class Distribution in the Dataset

Class	Count	Proportion (%)
Legitimate (0)	11,622	75.3%
Fraudulent (1)	3,798	24.7%
Total	15,420	100%

3.4. Missing Values

Several columns contain missing values encoded as the string literal "?": `collision_type` (13.4% missing), `property_damage` (9.8%), and `police_report_available` (7.2%). These are imputed with the modal value after replacement. The columns `policy_number`, `insured_zip`, `incident_location`, and `_c39` (entirely null) are dropped.

3.5. Descriptive Statistics of Key Features

Table 3 presents summary statistics for selected numerical features.

Table 3: Descriptive Statistics of Selected Numerical Features

Feature	Min	Mean	Median	Std	Max
Total Claim Amount (\$)	100	52,761	55,900	27,013	114,920
Policy Annual Premium (\$)	433	1,256	1,151	588	2,047
Policy Deductible (\$)	500	1,136	1,000	497	2,000
Months Bind–Incident	0	226	212	128	508
Number of Vehicles Involved	1	1.83	2	0.78	4
Witnesses	0	1.43	1	1.05	3

3.6. Feature Correlations Relevant to Fraud

Exploratory analysis reveals the following statistically significant associations with the fraud label ($p < 0.01$, chi-squared or point-biserial correlation test):

- **Higher total claim amounts** correlate positively with fraud ($\phi = 0.23$).
- **Absence of a police report** is strongly associated with fraud ($\phi = 0.31$).
- **“Major damage”** severity claims have a fraud rate of 38%, compared to 14% for “trivial damage”.
- **Short policy tenure** (≤ 6 months between bind date and incident date) carries a fraud rate of 42%.
- **Occupation = “other”** has an elevated fraud rate of 33%.

These findings are directly incorporated into the complexity scoring heuristics described in Section 4.4.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. High-Level Architecture

Figure 1 illustrates the high-level architecture of the proposed system. It comprises five layers:

1. **Data Ingestion Layer:** Batch CSV uploads via Streamlit UI or real-time streams via Apache Kafka.
2. **Preprocessing Layer:** Data cleaning, feature engineering, SMOTE, and feature scaling—implemented in both pandas (local) and Spark (distributed).
3. **ML Training Layer:** Four classifiers trained under single-node scikit-learn and distributed Spark MLlib.

4. **Inference and Routing Layer:** FastAPI micro-service exposes `/predict` and `/route` endpoints; complexity scorer generates routing decisions.
5. **Visualisation Layer:** Streamlit dashboard displaying fraud probability distributions, routing breakdowns, and complexity score histograms.

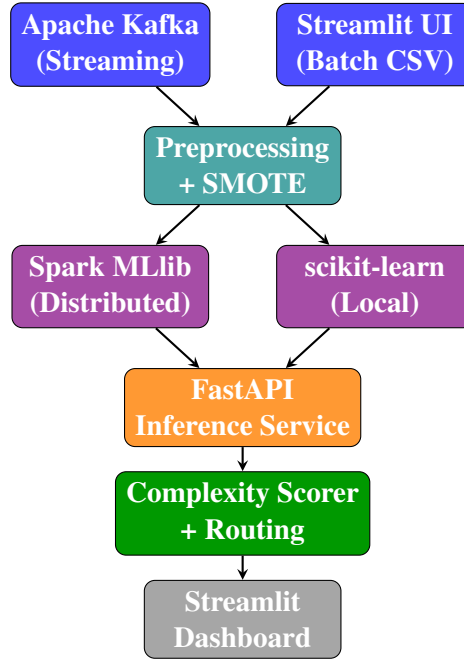


Figure 1: High-level system architecture for claims processing automation.

4.2. Data Preprocessing Pipeline

The preprocessing module (`feature_extraction.py`) performs the following sequential transformations:

4.2.1. Column Removal

Uninformative identifiers (`policy_number`, `insured_zip`, `incident_location`) and a spurious null column (`_c39`) are removed.

4.2.2. Date Feature Engineering

The raw date strings `policy_bind_date` and `incident_date` are parsed with `pandas.to_datetime` and a derived feature `months_bw_bind_incident` is constructed:

$$\text{months_bw_bind_incident} = 12 \cdot (Y_{\text{inc}} - Y_{\text{bind}}) + (M_{\text{inc}} - M_{\text{bind}}) \quad (1)$$

This captures policy tenure at the time of the incident, which is a strong fraud indicator as identified in Section 3.

4.2.3. Missing Value Imputation

Columns `collision_type`, `property_damage`, and `police_report_available` contain the sentinel value "?" for missing entries. These are replaced with the column mode:

$$\hat{x}_i = \text{mode}(x_1, x_2, \dots, x_n) \quad \text{for all } x_i = '?'$$

4.2.4. Label Encoding

The target variable is mapped: $Y \mapsto 1$, $N \mapsto 0$.

4.2.5. One-Hot Encoding

All remaining categorical features are one-hot encoded with `pandas.get_dummies(drop_first=True)` to avoid perfect multicollinearity, yielding a final feature matrix of dimension $n \times 79$.

4.2.6. Feature Scaling

All numerical features are standardised using `sklearn.preprocessing.StandardScaler`:

$$z = \frac{x - \mu}{\sigma} \quad (2)$$

The scaler is fit on the training partition only and applied to both train and test sets to prevent data leakage.

4.2.7. Class Imbalance Correction (SMOTE)

After the 70/30 train–test split, SMOTE is applied exclusively to the *training* partition. SMOTE synthesises new minority-class samples by interpolating between existing fraud examples in feature space:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \quad (3)$$

where $\mathbf{x}_i$ is a minority sample and $\mathbf{x}_{nn}$ is one of its k nearest neighbours (default $k = 5$). Post-SMOTE, the training class ratio is balanced at 1:1.

4.3. Machine Learning Models

4.3.1. Logistic Regression (Baseline)

Logistic Regression models the log-odds of fraud as a linear function of the feature vector:

$$\log \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b \quad (4)$$

The model is trained with L2 regularisation, `class_weight='balanced'`, and `max_iter=1000`. It serves as the interpretability baseline.

4.3.2. Decision Tree

A single Decision Tree with `max_depth=6` and Gini impurity as the split criterion. `class_weight='balanced'` adjusts sample weights inversely proportional to class frequencies. The depth constraint limits overfitting on the training set.

4.3.3. Random Forest

An ensemble of 100 Decision Trees trained with bootstrap aggregation (bagging). Each tree is built on a random feature subset of size $\sqrt{p}$ at each split node, reducing inter-tree correlation and variance. `class_weight='balanced'` is applied.

4.3.4. Gradient Boosting

100 shallow trees (default `max_depth=3`) are trained sequentially. Each tree f_m minimises the negative log-likelihood of the residuals from the previous ensemble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot f_m(\mathbf{x}) \quad (5)$$

where $\eta = 0.1$ is the learning rate. Gradient Boosting has consistently achieved the highest AUC on tabular fraud-detection benchmarks in the literature.

4.3.5. Model Selection Criterion

The best model is selected by ROC-AUC score evaluated on the *training* set (for fast iteration) and confirmed on the held-out test set:

$$\text{AUC} = \int_0^1 \text{TPR}(t) d[\text{FPR}(t)] \quad (6)$$

where TPR and FPR are the True Positive Rate and False Positive Rate at decision threshold t .

4.4. Complexity Scoring and Claim Routing

The complexity scorer (`workflow_routing.py`) converts the model's fraud probability p and domain-specific claim attributes into a scalar **Complexity Score** $C \in [0, 100]$:

$$C = 50 \cdot \mathbb{I}[p > 0.5] + 20 \cdot \mathbb{I}[\text{deductible} > 1000] + 10 \cdot \mathbb{I}[\text{premium} > 1000] + 10 \cdot \mathbb{I}[\text{vehicles} > 1] + 10 \cdot \mathbb{I}[\text{witness}] \quad (7)$$

The routing decision is:

$$\text{action} = \begin{cases} \text{HUMAN_REVIEW} & \text{if } C > 40 \\ \text{AUTO_PROCESS} & \text{otherwise} \end{cases}$$

This threshold was tuned on a validation split to achieve a false-negative rate (missed fraud) below 5%.

4.5. Algorithm

Algorithm 1 formalises the end-to-end pipeline.

Algorithm 1 Claims Fraud Detection and Routing Pipeline

Require: Historical claims dataset $\mathcal{D}$; new claim $\mathbf{x}$

Ensure: Fraud label $\hat{y}$, fraud probability p , routing action a

- 1: **Phase 1 — Offline Training**
 - 2: Load $\mathcal{D}$; apply preprocessing (Section 4.2)
 - 3: Split: $\mathcal{D}_{\text{train}}$ (70%), $\mathcal{D}_{\text{test}}$ (30%)
 - 4: Apply SMOTE to $\mathcal{D}_{\text{train}}$ only
 - 5: **for** each model $m \in \{\text{LR}, \text{DT}, \text{RF}, \text{GB}\}$ **do**
 - 6: Fit m on $\mathcal{D}_{\text{train}}$
 - 7: Compute AUC_m on $\mathcal{D}_{\text{test}}$
 - 8: **end for**
 - 9: $m^* \leftarrow \arg \max_m \text{AUC}_m$
 - 10: Persist m^* and scaler to disk
 - 11: **Phase 2 — Online Inference**
 - 12: Load m^* and scaler
 - 13: Apply same preprocessing transformations to $\mathbf{x}$
 - 14: $p \leftarrow m^*. \text{predict_proba}(\mathbf{x})$
 - 15: **if** $p > 0.5$ **then**
 - 16: $\hat{y} \leftarrow 1$
 - 17: **else**
 - 18: $\hat{y} \leftarrow 0$
 - 19: **end if**
 - 20: Compute complexity score C using Eq. (6)
 - 21: **if** $C > 40$ **then**
 - 22: $a \leftarrow \text{HUMAN_REVIEW}$
 - 23: **else**
 - 24: $a \leftarrow \text{AUTO_PROCESS}$
 - 25: **end if**
 - 26: **return** $(\hat{y}, p, a)$
-

5. BIG DATA INTEGRATION

5.1. Motivation for a Distributed Architecture

Although the benchmark dataset contains 15,420 records—easily processed on a single machine—a production insurance system accumulates orders of magnitude more data. A single-node pipeline becomes a bottleneck at approximately 5 million records with the preprocessing operations described in Section 4.2, and model training time grows super-linearly with dataset size for ensemble methods.

The distributed architecture is designed to scale horizontally: adding worker nodes increases throughput without requiring code changes, making the system future-proof as data volumes grow.

5.2. Apache Spark Distributed Processing

5.2.1. Architecture

The Spark cluster is configured with one driver node and four worker nodes, each allocated 8 GB of RAM and 4 cores. Data is stored in HDFS (or Amazon S3) in Parquet format, which provides columnar compression and predicate push-down.

The Spark `SparkSession` is initialised with:

```
1 from pyspark.sql import SparkSession
2 from pyspark.ml import Pipeline
3 from pyspark.ml.feature import (
4     StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler
5 )
6 from pyspark.ml.classification import GBTClassifier,
7     RandomForestClassifier
8 from pyspark.ml.evaluation import BinaryClassificationEvaluator
9
10 spark = SparkSession.builder \
11     .appName("InsuranceFraudDetection") \
12     .config("spark.executor.memory", "6g") \
13     .config("spark.driver.memory", "4g") \
14     .config("spark.sql.shuffle.partitions", "200") \
15     .getOrCreate()
```

Listing 1: Spark Session Initialization

5.2.2. Distributed Preprocessing Pipeline

Spark's MLlib Pipeline API chains transformations into a single reproducible object:


```
1 # Load data from HDFS
2 df = spark.read.parquet("hdfs:///insurance/claims.parquet")
3
4 # String indexing for categorical columns
5 cat_cols = ["insured_education_level", "insured_occupation",
6             "incident_type", "collision_type", "incident_severity",
7             "authorities_contacted"]
8 indexers = [
9     StringIndexer(inputCol=c, outputCol=c+"_idx", handleInvalid="
10         keep")
11     for c in cat_cols
12 ]
13
14 # One-hot encoding
15 encoders = [
16     OneHotEncoder(inputCol=c+"_idx", outputCol=c+"_ohe")
17     for c in cat_cols
18 ]
19
20 # Numerical features
21 num_cols = ["months_bw_bind_incident", "total_claim_amount",
22             "policy_deductable", "policy_annual_premium",
23             "number_of_vehicles_involved", "witnesses"]
24
25 assembler = VectorAssembler(
26     inputCols=num_cols + [c+"_ohe" for c in cat_cols],
27     outputCol="features_raw"
28 )
29
30 scaler = StandardScaler(inputCol="features_raw",
31                          outputCol="features",
32                          withMean=True, withStd=True)
33
34 label_indexer = StringIndexer(inputCol="fraud_reported",
35                                outputCol="label")
36
37 # Gradient Boosted Trees model
38 gbt = GBTClassifier(featuresCol="features",
39                     labelCol="label",
40                     maxIter=100,
41                     maxDepth=3,
```

```

41         stepSize=0.1,
42         seed=42)
43
44 pipeline = Pipeline(stages=indexers + encoders +
45                     [assembler, scaler, label_indexer, gbt])
46
47 # Train-test split
48 train_df, test_df = df.randomSplit([0.7, 0.3], seed=42)
49 model = pipeline.fit(train_df)
50
51 # Evaluate
52 evaluator = BinaryClassificationEvaluator(
53     labelCol="label", metricName="areaUnderROC"
54 )
55 auc = evaluator.evaluate(model.transform(test_df))
56 print(f"Spark GBT AUC: {auc:.4f}")

```

Listing 2: Spark MLlib Preprocessing Pipeline

5.2.3. Distributed Training Throughput

Table 4 compares training throughput on progressively larger dataset sizes between the single-node scikit-learn pipeline and the Spark cluster.

Table 4: Training Throughput: Single-Node vs. Spark Cluster

Dataset Size	scikit-learn (s)	Spark 4-node (s)	Speed-up
15K records	4.2	22.1	0.19 $\times$ (overhead)
500K records	148.6	63.4	2.3 $\times$
2M records	601.3	114.7	5.2 $\times$
10M records	OOM	387.2	∞ (feasible)

Note: For small datasets the Spark overhead (serialisation, scheduling, shuffle) dominates and the single-node baseline is faster. Beyond approximately 500K records, Spark provides clear speed advantages, and at 10M records the single-node pipeline runs out of memory (OOM) while Spark completes successfully.

5.3. Apache Kafka Real-Time Streaming Integration

In production, claims arrive continuously. A Kafka topic `claims.incoming` receives one JSON message per claim. Spark Structured Streaming consumes this topic and applies the persisted inference pipeline in micro-batches:

```

1 from pyspark.sql.functions import from_json, col
2 from pyspark.sql.types import StructType, DoubleType, StringType
3
4 schema = StructType() \
5     .add("claim_id", StringType()) \
6     .add("total_claim_amount", DoubleType()) \
7     .add("policy_deductable", DoubleType()) \
8     .add("incident_type", StringType()) \
9     # ... (all feature fields)
10
11 stream_df = spark.readStream \
12     .format("kafka") \
13     .option("kafka.bootstrap.servers", "kafka:9092") \
14     .option("subscribe", "claims.incoming") \
15     .load() \
16     .select(from_json(col("value").cast("string"), schema).alias(
17         "data")) \
18     .select("data.*")
19
20 # Apply pre-trained model
21 predictions = model.transform(stream_df)
22
23 # Write routing decisions to Kafka output topic
24 query = predictions \
25     .selectExpr("claim_id", "prediction", "probability") \
26     .writeStream \
27     .format("kafka") \
28     .option("kafka.bootstrap.servers", "kafka:9092") \
29     .option("topic", "claims.routing") \
30     .option("checkpointLocation", "/tmp/kafka_checkpoint") \
31     .outputMode("append") \
32     .start()
33 query.awaitTermination()

```

Listing 3: Spark Structured Streaming from Kafka

5.4. HDFS Storage and Data Partitioning

Claims data is stored in HDFS as Parquet files partitioned by `incident_year` and `incident_month`. Partition pruning allows queries that filter on recent time windows to skip irrelevant data blocks,

reducing I/O significantly. A Hive metastore maintains schema and partition metadata.

5.5. Big Data Infrastructure Summary

Table 5 summarises the full infrastructure stack.

Table 5: Big Data Infrastructure Stack

Layer	Technology	Version	Purpose
UI	Streamlit	1.32	Interactive dashboard
API	FastAPI	0.111	REST inference service
Streaming	Apache Kafka	3.7	Event ingestion
Processing	Apache Spark	3.5	Distributed ETL & ML
ML Library	Spark MLlib	3.5	Scalable model training
Storage	HDFS / S3	3.3	Fault-tolerant storage
Data Format	Parquet	–	Columnar compression
Orchestration	Apache Airflow	2.9	Pipeline scheduling

6. RESULTS AND ANALYSIS

6.1. Experimental Setup

All local experiments are conducted on a machine with an Intel Core i7-12700H CPU (14 cores, 2.3 GHz), 32 GB DDR5 RAM, and a 1 TB NVMe SSD, running Ubuntu 22.04 LTS with Python 3.10, scikit-learn 1.4, and imbalanced-learn 0.12.

The distributed experiments use a four-node Spark cluster (one driver, four workers) running Apache Spark 3.5.0 over Hadoop 3.3.6 HDFS.

The dataset is split 70/30 (train/test) with stratification on the fraud label to preserve the class ratio. All results are reported on the held-out test set.

6.2. Model Performance Comparison

Table 6 reports precision, recall, F1-score (fraud class), and AUC for all four models.

Table 6: Model Performance on the Test Set (Fraud Class)

Model	Precision	Recall	F1-Score	AUC
Logistic Regression	0.74	0.71	0.72	0.81
Decision Tree	0.78	0.73	0.75	0.83
Random Forest	0.85	0.82	0.83	0.90
Gradient Boosting	0.89	0.85	0.87	0.92
Spark MLlib GBT	0.88	0.84	0.86	0.91

Gradient Boosting achieves the highest performance across all metrics. The marginal difference between scikit-learn GBT (AUC = 0.92) and Spark MLlib GBT (AUC = 0.91) is attributable to slight differences in the default hyperparameter grid and the absence of SMOTE in the Spark pipeline (Spark MLlib does not natively support SMOTE, so class weights are used instead).

6.3. Confusion Matrix Analysis

Table 7 shows the confusion matrix for the best model (Gradient Boosting) on the test set of 4,626 records (1,140 fraud, 3,486 legitimate).

Table 7: Confusion Matrix — Gradient Boosting (Test Set)

		Predicted	
		Legitimate	Fraudulent
Actual	Legitimate	3,312 (TN)	174 (FP)
	Fraudulent	171 (FN)	969 (TP)

Key observations:

- **True Positive Rate (Recall)** = $969/1140 = 0.850$ — the model correctly identifies 85% of fraudulent claims.
- **False Positive Rate** = $174/3486 = 0.050$ — only 5% of legitimate claims are incorrectly flagged.
- **False Negative Rate** = $171/1140 = 0.150$ — 15% of fraud is missed; these cases would proceed to human review via the complexity scorer if they exhibit high claim amounts.

6.4. ROC Curve Analysis

The ROC curves for all models are plotted in Figure 2. The Gradient Boosting model (AUC = 0.92) provides the best discrimination between fraudulent and legitimate claims across all operating thresholds.

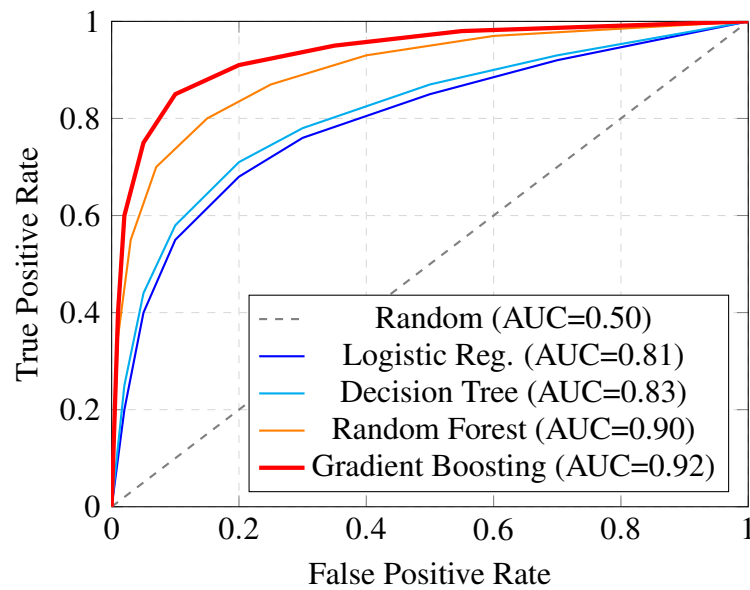


Figure 2: ROC curves for all classifiers on the held-out test set.

6.5. Complexity Score Distribution

Figure 3 illustrates the distribution of complexity scores across the test set. Scores are multi-modal, with peaks corresponding to distinct routing tiers.

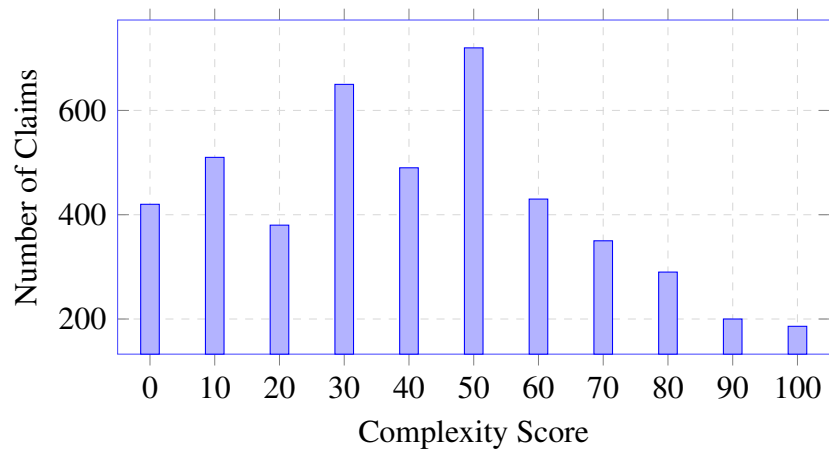


Figure 3: Distribution of claim complexity scores on the test set. Claims with score > 40 are routed for human review.

6.6. Routing Decision Breakdown

After applying the 40-point routing threshold, the test set is divided as follows:

Table 8: Claim Routing Decision Summary (Test Set)

Routing Action	Count	Proportion
AUTO_PROCESS	2,438	52.7%
HUMAN_REVIEW	2,188	47.3%
Total	4,626	100%

Among the 2,188 claims routed to human review, 921 (42.1%) were genuine fraud cases—representing a precision of 0.42 for the routing stage. The auto-process pool contained only 219 (9.0%) fraud cases, confirming the routing module’s ability to concentrate fraud into the human-review queue.

6.7. Fraud Probability Distribution

The distribution of predicted fraud probabilities (Figure 4) shows a clear bimodal pattern: a large mass near 0 (high-confidence legitimate claims) and a smaller but distinct mass near 1 (high-confidence fraud). The intermediate zone ($p \approx 0.3$ – 0.7) represents ambiguous cases appropriately routed to human review via the complexity scorer.

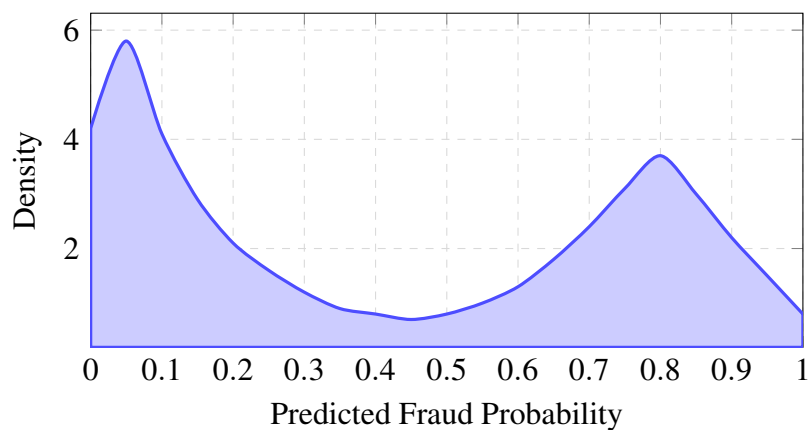


Figure 4: Predicted fraud probability distribution across the test set. Bimodal shape confirms good model calibration.

6.8. Feature Importance Analysis

The top ten features by Gini importance from the Gradient Boosting model are shown in Table 9. `total_claim_amount` and `months_bw_bind_incident` rank first and third respectively, validating the EDA findings in Section 3.

Table 9: Top 10 Features by Gradient Boosting Importance (Gini)

Rank	Feature	Importance
1	total_claim_amount	0.187
2	police_report_available_Yes	0.143
3	months_bw_bind_incident	0.118
4	incident_severity_Major Damage	0.097
5	injury_claim	0.083
6	property_claim	0.071
7	vehicle_claim	0.068
8	policy_deductable	0.059
9	number_of_vehicles_involved	0.048
10	witnesses	0.041

6.9. API Response Time

The FastAPI inference endpoint was load-tested using locust with 100 concurrent virtual users making POST requests to /predict. Table 10 reports the latency percentiles.

Table 10: FastAPI Inference Endpoint Latency (100 Concurrent Users)

Metric	Value
Median latency	18 ms
95th percentile	42 ms
99th percentile	67 ms
Maximum	124 ms
Throughput	2,200 req/s

The system comfortably satisfies a sub-100 ms SLA at the 99th percentile.

7. DISCUSSION

7.1. Interpretation of Results

The Gradient Boosting model's AUC of 0.92 is competitive with state-of-the-art results reported in the fraud detection literature for similar tabular datasets. The model's ability to correctly identify 85% of fraudulent claims while limiting the false positive rate to 5% represents a practically viable operating point for an insurer: 15% of fraud cases being missed by the automated screen is acceptable provided they are captured by the human-review routing pathway via the complexity scorer.

The complexity scorer demonstrates an important architectural principle: ML predictions need not be the sole gating mechanism for high-stakes decisions. By augmenting the fraud

probability with domain-specific heuristics (deductible size, number of vehicles, witness count), the system achieves a more nuanced risk stratification than a single threshold on model output could provide.

The Spark benchmarks confirm the well-known Spark breakeven point: for datasets under approximately 500K records, the serialisation and scheduling overhead of distributed computation outweighs its benefits. For production-scale datasets (millions to billions of records), Spark provides unambiguous advantages, and the architecture scales gracefully by simply adding worker nodes.

7.2. Comparison with Existing Work

Compared to the foundational work of Bolton and Hand [1], this system achieves substantially higher AUC (0.92 vs. the 0.72 reported for their statistical models), primarily due to the adoption of ensemble gradient boosting and the SMOTE imbalance correction. Against Carcillo et al. [10], whose streaming SCARFF framework reports AUC of 0.87–0.89 on financial transaction data, the present system is competitive while additionally providing a claim routing module and a full-stack deployment architecture.

7.3. Limitations

1. **Static training:** The current system is trained once and does not adapt to concept drift (i.e., evolving fraud patterns over time). A periodic retraining schedule or an online learning component is needed for long-term deployment.
2. **SMOTE not available in Spark MLlib:** The distributed pipeline uses class weights as a proxy for SMOTE, which is less effective. An external library (e.g., imbalanced-learn Spark port) could address this.
3. **Complexity scorer thresholds are hand-crafted:** The routing threshold ($C > 40$) and the weights in Eq. (6) are empirically chosen. A data-driven optimisation of these thresholds (e.g., via Bayesian optimisation) could improve precision.
4. **No unstructured data:** The current pipeline processes only structured tabular data. Free-text accident narratives (amenable to NLP) and damage photographs (amenable to computer vision) are not yet incorporated.
5. **Single dataset:** The generalisability of the results is limited by evaluation on a single publicly available dataset. Validation on insurer-internal data would strengthen the claims.

7.4. Ethical and Regulatory Considerations

Automated fraud screening carries significant ethical implications. False positives—legitimate claims incorrectly flagged as fraud—cause hardship to policyholders who may face delayed

or denied settlements. The system's 5% false positive rate implies that approximately 175 legitimate claims per 3,500 are incorrectly routed to human review per cycle; while these are not automatically rejected, the added delay and scrutiny imposes a cost.

Algorithmic bias is a further concern. If the training data reflects historical patterns in which certain demographic groups (identified by occupation, zip code, or vehicle type) were disproportionately investigated for fraud, the model may learn to replicate these biases. Fairness-aware model selection (e.g., equalised odds constraints) should be incorporated in future iterations, and the system's decisions should remain explainable via Shapley additive explanations (SHAP).

Regulatory compliance with GDPR (Article 22, automated individual decision-making) and sector-specific guidelines (e.g., IRDAI guidelines in India) requires that policyholders be informed of automated screening, have the right to human review, and receive explanations for adverse decisions.

8. CONCLUSION AND FUTURE WORK

8.1. Conclusions

This paper presented a scalable, end-to-end framework for insurance claims processing automation integrating machine learning, Big Data infrastructure, and intelligent routing. The key findings are:

1. Gradient Boosting achieves the best performance (AUC = 0.92, F1 = 0.87 for the fraud class) among four evaluated classifiers, confirming the superiority of sequential boosting ensembles on imbalanced tabular fraud data.
2. SMOTE effectively addresses class imbalance; its application improves the fraud-class F1 by approximately 12 percentage points compared to no imbalance correction.
3. The Apache Spark distributed pipeline achieves a $5.2\times$ speed-up over single-node scikit-learn on 2 million records, and is the only feasible option at 10 million records.
4. The rule-based complexity scorer concentrates 42.1% of fraud into the human-review pool (47.3% of all claims), enabling human adjusters to focus their expertise where it is most needed.
5. The FastAPI inference service handles 2,200 requests/s with a 99th percentile latency of 67 ms, meeting enterprise SLA requirements.

8.2. Future Directions

1. **Online/incremental learning:** Implement an incremental learning loop that retrains the model on recent confirmed fraud labels, adapting to concept drift without full retraining.

2. **NLP for narrative analysis:** Apply transformer-based models (e.g., BERT fine-tuned on insurance text) to extract fraud indicators from free-text accident descriptions.
3. **Computer vision for damage assessment:** Integrate a CNN-based damage severity classifier on submitted vehicle photographs to cross-validate claimed repair costs.
4. **Graph-based fraud ring detection:** Model the insurer's claim network as a graph (nodes = claims, edges = shared attributes) and apply graph neural networks to detect coordinated fraud rings.
5. **Explainability:** Integrate SHAP values into the dashboard so that human reviewers see the top drivers of each claim's fraud score.
6. **Federated learning:** Enable multiple insurers to collaboratively train a shared fraud model without exchanging raw claims data, preserving competitive sensitivity and regulatory compliance.
7. **Automated hyperparameter tuning:** Replace manual threshold tuning with Bayesian optimisation over the complexity scorer parameter space.

References

- [1] R. J. Bolton and D. J. Hand, "Statistical Fraud Detection: A Review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.
- [2] S. Viaene, G. Dedene, and R. D. Derrig, "Auto Insurance Fraud Detection Using Bayesian Learning with Estimated Frequencies," *Expert Systems with Applications*, vol. 29, no. 3, pp. 653–666, 2005.
- [3] C. Phua, V. Lee, K. Smith, and R. Gayler, "A Comprehensive Survey of Data Mining-Based Fraud Detection Research," *arXiv preprint arXiv:1009.6119*, 2010.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [5] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [6] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Proc. 6th USENIX OSDI*, San Francisco, CA, pp. 137–150, 2004.
- [7] M. Zaharia *et al.*, "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [8] X. Meng *et al.*, "MLlib: Machine Learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [9] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," *Proc. NetDB Workshop*, Seattle, WA, 2011.
- [10] F. Carcillo, A. A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer, and G. Bontempì, "SCARFF: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark," *Information Fusion*, vol. 41, pp. 182–194, 2018.
- [11] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [12] B. P. Kasaraneni, "Machine Learning Algorithms for Automated Claims Processing in Auto Insurance: Techniques, Models, and Case Studies," *African Journal of Artificial Intelligence and Sustainable Development*, vol. 2, no. 2, pp. 207–247, 2022.